Stats 21 - HW 3

Nitin Kunaparaju

Homework copyright Miles Chen. Problems have been adapted from the exercises in Think Python 2nd Ed by Allen B. Downey.

The questions have been entered into this document. You will modify the document by entering your code.

Make sure you run the cell so the requested output is visible. Export it as an HTML file and then print to PDF.

Homework is an opportunity to practice coding and to practice problem solving.

Doing exercises is where you will do most of your learning.

Using AI or copying someone else's solutions takes away your learning opportunities. It is also academic dishonesty.

Reading

- Chapter 10 - Dictionaries
- Chapter 12 - Tuples
- Chapter 13 - Files

Please keep up with the reading. The chapters are short.

Modified Exercise based on Think Python 2ed. Chap 11, Section 10, Exercise 11.1

https://greenteapress.com/thinkpython2/html/thinkpython2012.html

Write a function that reads the words in `words.txt` and stores them as keys in a dictionary. It doesn't matter what the values are. Then you can use the `in` operator as a fast way to check whether a string is in the dictionary.

In [150…

```
def make_word_dict():
    t = {}
    fin = open("words.txt")
    for line in fin:
        word = line.strip()
```

```
        t[word] = 0

    return t

word_dict = make_word_dict()

"hello" in word_dict
```

Out[150… True

Modified Exercise based on Think Python 2ed. Chap 11, Section 10, Exercise 11.4

https://greenteapress.com/thinkpython2/html/thinkpython2012.html

In HW2 Exercise 10.7, you created a function called `has_duplicates()`. It takes a list as a parameter and returns `True` if there is any object that appears more than once in the list.

Use a dictionary to write a faster, simpler version of `has_duplicates()`.

In [151…

```
def has_duplicates(t):
    dup_dict = {}
    for item in t:
        if item in dup_dict:
            return True
        else:
            dup_dict[item] = 0
    return False
```

In [152…

```
has_duplicates(['a','b','b','c'])
```

Out[152… True

In [153…

```
has_duplicates(['a','b','c','a'])
```

Out[153… True

In [154…

```
has_duplicates(['a', 'b', 'c', 'd'])
```

Out[154… False

Modified Exercise based on Think Python 2ed. Chap 11, Section 10, Exercise 11.5

https://greenteapress.com/thinkpython2/html/thinkpython2012.html

A Caesar cipher is a weak form of encryption that involves 'rotating' each letter by a fixed number of places. To rotate a letter means to shift it through the alphabet, wrapping around to the beginning if necessary. "A" rotated by 3 is "D". "Z" rotated by 1 is "A".

Two words are "rotate pairs" if you can rotate one of them and get the other. For example, "cheer" rotated by 7 is "jolly".

Write a script that reads in the wordlist `words.txt` and finds all the rotate pairs of words that are 5 letters or longer.

One function that could be helpful is the function `ord()` which converts a character to a numeric code. Keep in mind that numeric codes for uppercase and lowercase letters are different.

Some hints:

- you can write helper functions, such as a function that will rotate a letter by a certain number and/or another function that will rotate a word by a number of letters
- to keep your script running quickly, you should use the wordlist dictionary from exercise 11.1

In [155…

```
# write your code here.

def make_word_list():
    """Reads lines from a file and builds a list."""
    t = []
    fin = open('words.txt')
    for line in fin:
        word = line.strip()
        t.append(word)
    return t
word_list = make_word_list()

def rotate_word(word, num):
    rotated_word = ""
    for letter in word:
        if 'a' <= letter <= 'z':
            rotated_word += chr((ord(letter) - ord('a') + num) % 26 +ord('a'))
        elif 'A' <= letter < 'Z':
             rotated_word += chr((ord(letter) - ord('a') + num) % 26 +ord('a')).low
        else:
            rotated_word += letter
    return rotated_word



def rotate_pairs(words_list):
    rotated_pairs = []
    for word in words_list:
        for i in range(1, 26):
            if(len(word) >= 5 and rotate_word(word, i) in word_dict):
                rotated_pairs.append((word, rotate_word(word, i)))
    return rotated_pairs

#rotate_pairs(word_list)

hi = sorted("hello")
```

```
print(hi)
```

```
['e', 'h', 'l', 'l', 'o']
```

Modified Exercise based on Think Python 2ed. Chap 12, Section 10, Exercise 12.2

https://greenteapress.com/thinkpython2/html/thinkpython2013.html

Write a program that reads a word list from a file (see Section 9.1) and finds all the sets of words that are anagrams.

After finding all anagram sets, print the list of all anagram sets that have 6 or more entries in it.

Here is an example of what the output might look like:

```
['abets', 'baste', 'bates', 'beast', 'beats', 'betas', 'tabes']
['acers', 'acres', 'cares', 'carse', 'escar', 'races', 'scare',
'serac']
['acred', 'arced', 'cadre', 'cared', 'cedar', 'raced']
...
```

Hint:

First traverse the entire wordlist to build a dictionary that maps from a collection of letters to a list of words that can be spelled with those letters. The question is, how can you represent the collection of letters in a way that can be used as a key? i.e. The word "eat" and the word "tea" should be in a list associated with a key ('a','e','t').

In [156…

```
"""
def counts_dict(word):
    dict = {}
    for letter in word:
        if letter not in dict:
            dict[letter] = 1
        else:
            dict[letter] += 1
    return dict

def anagrams(letters):
    anagram_list = []
    for word in word_list:
        if(counts_dict(word) == counts_dict(letters)):
            anagram_list.append(word)
    return anagram_list

def is_anagram(word1, word2):
    word1 = word1.replace(" ", "").lower()
    word2 = word2.replace(" ", "").lower()
```

```
    return sorted(word1) == sorted(word2)
"""

def letters_dict(words_list):
    letter_dict = {}
    for word in words_list:
        sorted_word = tuple(sorted(word))
        if sorted_word in letter_dict:
            letter_dict[sorted_word].append(word)
        else:
            letter_dict[sorted_word] = [word]

    return letter_dict

words_dict = letters_dict(word_list)
print(words_dict[('a', 'c', 'r')])
```

```
['arc', 'car']
```

NumPy Exercises

For the following exercises, be sure to print the desired output. You will not receive credit for problems that do not print the desired output.

Some of these exercises will require the use of functions in numpy that may not have been covered in class. Look up documentation on how to use them. I always recommend checking the official reference at https://numpy.org/doc/stable/reference/index.html

In [157…

```
import numpy as np
rng = np.random.default_rng(seed = 1)
```

Exercise NP.1:

Task: Create an array `b` of 10 random integers selected between 0-99

Desired output: `[47 51 75 95  3 14 82 94 24 31]` of course yours might be different

In [158…

```
b = rng.integers(0, 99, size = 10)
b
```

Out[158…

```
array([46, 50, 74, 94,  3, 14, 81, 93, 24, 30])
```

NP.1b:

Task: reverse the elements in `b`. Hint: Try slicing the array, but backwards

In [159…

```
backwards_b = b[::-1]
backwards_b
```

Out[159…

```
array([30, 24, 93, 81, 14,  3, 94, 74, 50, 46])
```

NP.2a:

Task: Create an array `c` of 1000 random values selected from a normal distribution centered at 100 with sd = 15, rounded to 1 decimal place. Print only the first 10 values.

Desired output: `[ 92.1  83.9 113.   65.5 126.2  88.6 104.8  96.3 121.9  69.1]` of course your values may be different

In [160…

```
c = rng.normal(100, 15, (1000))
c.round(1, c)
c[0:10]
```

Out[160…

```
array([106.7,  91.9, 108.7, 105.5, 104.4, 100.4, 108.2,  89. ,  97.6,
        92.8])
```

NP.2b:

Perform a Shapiro-Wilk test to see if the values in c come from a normal distribution. Report the p-value and appropriate conclusion.

Look up `scipy.stats.shapiro` for usage and details.

https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.shapiro.html

In [161…

```
from scipy.stats import shapiro

stat, p_val = shapiro(c)

alpha = 0.05
if(p_val >= alpha):
    print("We fail to reject the null hypothesis because the p-value " + str(p_val)
else:
    print("We reject the null hypothesis because the p-value " + str(p_val) + " is
```

```
We fail to reject the null hypothesis because the p-value 0.7866973956692646 is grea
ter than alpha = 0.05
```

NP.2c:

Identify and print the values in `c` that are more than 3 standard deviations from the mean of `c`. Report the proportion of values that are more than 3 sd from the mean.

Desired output: `[ 54.1 ... 148.3 ]`

`0.32 of the values are beyond 3 sd from the mean.`

In [162…

```
sd_c = []
for value in c:
    if(value < 100):
        if(value + 45 < 100):
            sd_c.append(value)

    elif(value > 100):
        if(value - 45 > 100):
```

```
            sd_c.append(value)

print(sd_c)
print(str(len(sd_c) / len(c)) + " of the values are beyond 3 sd form the mean")
```

```
[np.float64(146.5), np.float64(53.8), np.float64(46.8), np.float64(156.3)]
0.004 of the values are beyond 3 sd form the mean
```

NP.3:

Task: Make a 3x3 identity matrix called `I3`

Desired output:

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

In [163…

```
I3 = np.array([[1.,0.,0.], [0., 1., 0.], [0., 0., 1.]])
I3
```

Out[163…

```
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

NP.4a:

Task: Make a 10x10 array of values 1 to 100. Call it X

Desired output:

```
[[  1   2   3   4   5   6   7   8   9  10]
 [ 11  12  13  14  15  16  17  18  19  20]
 [ 21  22  23  24  25  26  27  28  29  30]
 [ 31  32  33  34  35  36  37  38  39  40]
 [ 41  42  43  44  45  46  47  48  49  50]
 [ 51  52  53  54  55  56  57  58  59  60]
 [ 61  62  63  64  65  66  67  68  69  70]
 [ 71  72  73  74  75  76  77  78  79  80]
 [ 81  82  83  84  85  86  87  88  89  90]
 [ 91  92  93  94  95  96  97  98  99 100]]
```

In [164…

```
x = np.reshape(np.arange(1, 101), (10, 10))
x
```

Out[164…

```
array([[  1,   2,   3,   4,   5,   6,   7,   8,   9,  10],
       [ 11,  12,  13,  14,  15,  16,  17,  18,  19,  20],
       [ 21,  22,  23,  24,  25,  26,  27,  28,  29,  30],
       [ 31,  32,  33,  34,  35,  36,  37,  38,  39,  40],
       [ 41,  42,  43,  44,  45,  46,  47,  48,  49,  50],
       [ 51,  52,  53,  54,  55,  56,  57,  58,  59,  60],
       [ 61,  62,  63,  64,  65,  66,  67,  68,  69,  70],
       [ 71,  72,  73,  74,  75,  76,  77,  78,  79,  80],
       [ 81,  82,  83,  84,  85,  86,  87,  88,  89,  90],
       [ 91,  92,  93,  94,  95,  96,  97,  98,  99, 100]])
```

NP.4b:

Task: Make a copy of X, call it Y (1 line). Replace all values in Y that are not squares with 0 (1 line). see `numpy.isin()`

Desired output:

```
[[  1   0   0   4   0   0   0   0   9   0]
 [  0   0   0   0   0  16   0   0   0   0]
 [  0   0   0   0  25   0   0   0   0   0]
...
 [  0   0   0   0   0   0   0   0   0 100]]
```

In [165…

```
y = x.copy()
squares_100 = [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

NP.5:

Task: Use `np.tile()` to tile a 2x2 diagonal matrix of integers to make a checkerboard pattern. Call the matrix `checkers`

Desired output:

```
[[1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 ...
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]]
```

In [166…

```
np.tile([[1, 0],[0,1]], (4, 4))
```

Out[166…

```
array([[1, 0, 1, 0, 1, 0, 1, 0],
       [0, 1, 0, 1, 0, 1, 0, 1],
       [1, 0, 1, 0, 1, 0, 1, 0],
       [0, 1, 0, 1, 0, 1, 0, 1],
       [1, 0, 1, 0, 1, 0, 1, 0],
       [0, 1, 0, 1, 0, 1, 0, 1],
       [1, 0, 1, 0, 1, 0, 1, 0],
       [0, 1, 0, 1, 0, 1, 0, 1]])
```

NP.6:

Task: convert the values in `f_temp` from Farenheit to celsius. The conversion is subtract 32, then multiply by 5/9. Round to 1 decimal place.

Desired output:

```
[[21.1 21.7 22.2 22.8 23.3 23.9]
 ...
 [34.4 35.  35.6 36.1 36.7 37.2]]
```

In [167…

```
# do not modify
f_temp  = np.arange(70,100).reshape((5,6))
```

In [168…

```
c_temp = np.multiply(np.subtract(f_temp, 32), (5/9)).round(1)
c_temp
```

Out[168…

```
array([[21.1, 21.7, 22.2, 22.8, 23.3, 23.9],
       [24.4, 25. , 25.6, 26.1, 26.7, 27.2],
       [27.8, 28.3, 28.9, 29.4, 30. , 30.6],
       [31.1, 31.7, 32.2, 32.8, 33.3, 33.9],
       [34.4, 35. , 35.6, 36.1, 36.7, 37.2]])
```

NP.7:

Task: Convert values in the matrix `x` into z-scores by column, call it matrix `z`. That is: each column should have a mean of 0 and std of 1. (subtract the column mean, and divide by the column std). (not required, but see if you can do this in one line)

Print the column means and column std to show that they have been standardized.

Desired output:

```
[[ 0.41148428  1.04307072 -1.43031951]
 [-0.34069129 -1.14825432 -0.77270135]
...
 [-0.60705751 -1.08536687 -0.57430135]
 [-1.28156585 -0.81250928  1.52877401]]

[-1.16573418e-16  7.77156117e-17  1.44328993e-16]
[1. 1. 1.]
```

In [169…

```
# do not modify
rng = np.random.default_rng(seed = 100)
x = rng.integers(0, 100, size = 30).reshape(10,3)
x
```

Out[169…
```
array([[76, 83, 12],
       [59,  8, 28],
       [44,  4, 58],
       [97, 94, 59],
       [41, 79, 92],
       [91,  0, 68],
       [71, 18, 50],
       [98, 68, 28],
       [32, 62, 14],
       [58, 57, 59]])
```

In [170…
```
col_means = np.mean(x, axis = 0)
col_std = np.std(x, axis = 0)
z = ((x-col_means) / col_std)
z
np.std(z, axis = 0)
np.mean(z, axis = 0)
```

Out[170…
```
array([-1.16573418e-16,  7.77156117e-17,  1.44328993e-16])
```

NP.8:

Task: Convert values in the matrix `x` into scaled values from 0 to 10. That is take each column and scale values linearly so that the largest value is 10, and the smallest value in the column is 0. Round results to 2 decimal places. Call the result `y`

(Not required, but see if you can do the calculations in one line.)

Desired output:

```
[[0.67 0.88 0.  ]
 [0.41 0.09 0.2 ]
 ...
 [0.   0.66 0.02]
 [0.39 0.61 0.59]]
```

In [171…
```
# do not modify
rng = np.random.default_rng(seed = 100)
x = rng.integers(0, 100, size = 30).reshape(10,3)
x
```

Out[171…
```
array([[76, 83, 12],
       [59,  8, 28],
       [44,  4, 58],
       [97, 94, 59],
       [41, 79, 92],
       [91,  0, 68],
       [71, 18, 50],
       [98, 68, 28],
       [32, 62, 14],
       [58, 57, 59]])
```

In [172…
```
y = np.divide(x, 10).round(2)
y
```

Out[172…
```
array([[7.6, 8.3, 1.2],
       [5.9, 0.8, 2.8],
       [4.4, 0.4, 5.8],
       [9.7, 9.4, 5.9],
       [4.1, 7.9, 9.2],
       [9.1, 0. , 6.8],
       [7.1, 1.8, 5. ],
       [9.8, 6.8, 2.8],
       [3.2, 6.2, 1.4],
       [5.8, 5.7, 5.9]])
```

NP.9:

Task: Replace all NaN values in the matrix `x` with 0.

Desired output:

```
[[76. 83. 12.]
 [59.  8.  0.]
 [44.  0. 58.]
 [ 0. 94. 59.]]
```

In [173…
```
# do not modify
rng = np.random.default_rng(seed = 100)
x = rng.integers(0, 100, size = 12).reshape(4,3).astype('float')
row = np.array([1, 2, 3])
col = np.array([2, 1, 0])
x[row, col] = np.nan
x
```

Out[173…
```
array([[76., 83., 12.],
       [59.,  8., nan],
       [44., nan, 58.],
       [nan, 94., 59.]])
```

In [174…
```
np.nan_to_num(x, nan = 0)
```

Out[174…
```
array([[76., 83., 12.],
       [59.,  8.,  0.],
       [44.,  0., 58.],
       [ 0., 94., 59.]])
```